ПРОГРАММИРОВАНИЕ

Дополнительные главы C++

на второй паре...

Спецкурс с этой пары

1 Микропроцессы

1.1 Процессор и время выполнения

- Процессор (логический процессор).
- Синхронное устройство с тактовой частотой.
- Каждый такт выполняется операция.

$$2.4 \text{ ГГц} = 2.4 \cdot 10^9$$

$$c = \text{число тактов на инструкцию}$$

$$T = \frac{1}{f} \quad \text{- время одного такта}$$

$$c \cdot T = \text{время на 1 инструкцию}$$

$$I = \text{штук инструкций}$$

$$I \cdot c \cdot T = \text{время работы программы}$$

Инструкция - вычисления интерполяционного многочлена Лагранжа - раньше были как пример в процессорах моторолла.

Чем больше инструкция, тем больше тактов требовалось под инструкцию.

Микрокод - подпрограмма внутри процессора.

```
double *a;
a[i];
*(a + i * 8);
```

1.2 RISC и CISC

Сорок лет назад другой вариант:

- Оставим наипростейшие инструкции.
- А они будут воссоздавать инструкции процессора.
- Reduced Instruction Set Computer.
- Complete Instruction Set Computer.
- i486 1.2млн. транзисторов.
- A Sun - 56тыс. тр.
- Примитивные инструкции ровно за 1 такт.

Когда меньше транзисторов, они меньше греются -> подняли тактовую частоту:

- $I +$
- $C = 1$
- T -

Sun - уделал рынок серверов в то время.

1. Считать инструкцию из памяти (выбор).
   - У инструкций intel переменный размер.
   - У RISC все инструкции - одного размера - 4 байта.
2. Декодировать.
   - Код инструкции - операнды: `r_a`, `r_b`, `r_c` - регистры, могут быть внутри и константы маленькие от 0 до 31.

1.3 Структура инструкций

Операнды в памяти - **медленно**.

- load/store - память можно записать/считать.
- Число методов адресации:
  - Motorola - 18 - безумно медленно.
  - `Intel - 2 reg \ reg + const`.

Пример:

```
*p
*(p + 1)
```

1.3.1 Регистры

Увеличено число регистров:

- 32 целых.
- 32 с плавующей точкой.

Более эффективно можно использовать повторно регистры.

Регистр - ячейка памяти внутри процессора, которая упоминается в инструкции, базовая ячейка.

```
x++;
```

Процессор увидит:

```
x -> r_0
r_0 + 1 -> r_1
r_1 -> x
```

1.3.2 Устройства процессора

MMU (Memory managment Unit) - обработчик load / store.

Вычисления:

- ALU (Arithmetic Logical Unit).
- IU (Integer Unit).
- FPU (Floating Point Unit) - последние 30 лет.

За счёт разделения они могут работать параллельно.

Пока что 4 такта, не 1, зато параллельные.

1.4 Конвейер

1.4.1 Стадии конвейера

1. Выборка.
2. Декодирование.
3. Исполнение.
4. Выполнение.

Раз за разом. Друг против друга. Значит теперь:

```
Выборка
Декодирование   Выборка
Исполнение      Декодирование
Выполнение      Инструкция
                Запись
```

И так далее, все устройства заняты и работают параллельно.

По аналогии с автостроение - назвали конвейер. Работает в 4 раза быстрее. Получается $C = 1$!

После стали разделять на части отдельные части Unit. Получилось разделить примерно на 10 - 12 стадий. До 16 - 18.

1.4.2 Зависимые инструкции

Как написать программу так, чтобы конвейер не работал?

Зависимые инструкции. Каждая зависит от следующей:

```
a = b;
c = a + d;
```

Компилятор превратит в:

```
a = b
c = b + d;
```

Но например вызов функции он так убрать не может!

```
a = f(b)
c = a + d
```

```
a = b + c;
d = a * 2;
```

```
b -> r_0
c -> r_1
r_0 + r_1 -> r_2
```

> **Простой конвейера, `r_2` не готов.**

```
r_2 * 2 -> r_3
r_3 -> d
```

1.4.3 Переупорядочивание инструкций

Пример:

```
i++;
j++;
a = b + i;
d = a * 2;
```

Компилятор:

```
i -> r_0
j -> r_1
b -> r_2
c -> r_3
r_0 + 1 -> r_4
r_1 + 1 -> r_5
r_2 + r_3 -> r_6
r_4 -> i
r_5 -> j
r_6 * 2 -> r_7
...
```

Компиляторы сильно продвинулись.

1.5 Команды перехода

1. вызов функции (Branch Unit);
2. Циклы;
3. if (Компилятор не знает куда мы пойдём).

Пример:

```
double f (double *a, int n)
{
  double s = 0; int i;
  for (i = 0; i < n ; i++)
```

```
    s+= a[i];
  return s;
}
```

```
f_0 = 0 // s = 0
r_2 = 0 // i
r_0 = a
r_1 = n
L2
cmp r_2, r_1 /// cmp i, n
если >= 0 goto L1 // Выход из цикла
r_3 = r_2 * 8 // i * 8
f_1 = *(r_0 + r_3)
f_0 = f_0 + f_1
r_2 = r2 + 1
goto L2
L1
```

1.5.1 Предсказание переходов

Чтобы циклы работали используется „“предсказание’„ переходов.

- Примитивное предсказание, что переход к старшим не состоится, а к младшим состоится.
- $n - 1$ раз он угадает, в последнем ошибётся.

```
if (a > b)
  max = a;
else
  max = b;
```

```
if (a != 0)
  b = 1/a
else
  b = 0;
```

Компилятор:

```
a -> r_0
cmp r_0, 0
если равно то нужно перейти в L1
Иначе
1 / r_0 -> r_1
goto L2
L1:
0 -> r_1
L2: ответ в r_1
```

Выполняется первое после if.

> **То есть первым перед if стоит ставить наиболее вероятное действие!**

1.5.2 Разворачивание цикла

Вернёмся к:

```
double f (double *a, int n)
{
  double s = 0; int i;
  for (i = 0; i < n ; i++)
    s+= a[i];
```

```
  return s;
}
```

Компилятор часто меняет маленький код:

```
double f (double *a, int n)
{
  double s = 0;
  int i, m = n & 7;
  for (i = 0; i < m ; i++)
    s += a[i];

  for (; i < n ; i += 8)
    s += a[i]
       + a[i + 1]
       + a[i + 2]
       + a[i + 3]
       + a[i + 4]
       + a[i + 5]
       + a[i + 6]
       + a[i + 7];

  return s;
}
```

1.5.3 Как сломать оптимизацию

Нарпимер ООП.

```
double g (double x)
{ return x * x; }

double f (double *a, int n)
{
  double s = 0; int i;
  for (i = 0 ; i < n ; i++)
    s += g(a[i]);
  return s
}
```

Теперь не раскроет!

1.5.4 PowerPC

В ЦП PowerPC есть набор битов для предсказания переходов которые компилятор проставляет.

P.S. Правда компиляторов нет...

```
a > b
a > c
a > b && b > c
```

1.5.5 ARM

ЦП ARM отличились. Каждая инструкция является условной...

Для каждой зарезервировали 4 бита, которые говорят когда её исполнять. По-умолчанию 0000, то есть всегда.

```
if (a > b)
  max = a;
```

```
else
  max = b;
```

```
a -> r_0
b -> r_1
cmp r_0, r_1
mov >  r_0 -> r_3
mov <= r_1 -> r_3
```

Она пройдёт выборку, декодирование, но может не использоваться. Но получается без goto. Потеряли несколько битов.

1.6 Преобразование CISC-инструкций

В середине 80х Intel стал по скорости проигрывать RISC. Что делать?

Из-за microsoft так у intel:

1. инстр CISC;
2. Потом блок преобразующий в RISC конструкции;
3. и потом обработка.

В 00х AMD ввела 64 битные инструкции и дала доступ в RISC напрямую.

1.7 Несколько конвейеров

Со временем появились ЦП с 2мя конвейерами.

Стоит диспетчер направляющий в различные конвейеры. Например два IU и один общий FPU на конвейер. Со временем и FPU в каждый конвейер.

```
I6, I5, I4, I3, I2, I1 ->
1: I1, I3, I5
2: I2, I4, I6
-> I1, I2, I4, I6 - исполнение не по порядку (больше хлапот)
```

> **Не понял:**
> - `0x1000` - данные на диске.
> - `0x100` - блок данных для записи на диск.

Появились отдельные инструкции: исполнить по порядку. Которая вырубает конвейер.

```
if (x != 0)
  a = 1/x
else
  a = 0;
```

1.8 Разрыв между скоростью процессора и памяти

- В 87м при появлении RISC.
- Частота процессора кратно стала больше частоты памяти.
- А инструкции он берёт из памяти.

В 80м году - 16 МГц скорость.

Сейчас - 20 МГц.

Прогресса нет.

1.9 Как устроена память

1.9.1 На конденсаторах (нанотоках) DRAM

- Память на основе конденсатора, заряжен / разряжен.
- 18 раз в секунду память регенерируется.
- Конденсатор дольше держит ток, дольше переключается, меньше держит, чаще зарядать нужно.
- Dynamic.

1.9.2 На транзисторах (больших токах) SRAM

- Будет мгновенно переключать ток.
- Static.

1.9.3 Что делать со скоростью памяти? Кеш память!

- Сохраняет последние обращения памяти внутри процессора.
- SRAM.
- Кеш инструкций.
- Кеш данных.

Как узнать, что инструкция, а что данные?

Только после того как их прочли (внутрь обработки инструкции).

Unified кеш - Общий кеш инструкций и данных.

1. L_1 - f (frequency) - Кеш инструкций и данных (два разных).
2. L_2 - $\frac{f}{5}$ - Содержит любую информацию считанную из внешней памяти (больше и объединён).
3. L_3 - $\frac{f}{10}$ - Общая память между ядрами (Для взаимообмена).

Объёмы, пример для серверов:

- L1i: 5.3 MB $\cdot$ 168 cores.
- L1d: 5.3 MB $\cdot$ 168 c.
- L2: 168 MB $\cdot$ 168 c.
- L3: 768 MB $\cdot$ 24.

1.10 Как устроена кеш память?

Таблица - адрес и значение.

1.10.1 Что такое адрес?

Число.

- Физический адрес - номер ячейки, только ОС.
- Логический (Виртуальный) адрес.

Таблица: Виртуальный - Физический.

Но не каждого байта, а кусков, по 4 КБ.

> **СДЕЛАТЬ:**
> ОПИСАТЬ ПЕРЕВОД ВИРТУАЛЬНОГО АДРЕСА В ФИЗИЧЕСКИЙ

Виртуальный нужен для того, чтобы один процесс не мог разрушить данные другого.

Когда происходит обращение в памяти на блок не предназначенный для процесса:

- Segmentation fault.

Когда происходит обращение в памяти на не существующий блок в памяти, вне таблицы:

- Bus Error.

КЕШ - физическое устройство, которое работает с физическими объектами.

1.10.2 Поиск значения

Как найти щначение:

1. Берем номер k.
2. Идём в k ячейку, если значение совпало, то данные есть.

```
64 - b - i
```

Обычно:

- 4 КБ;
- $i + b = 12$;
- $i = 7$;
- 128 строчек в таблице;
- длина строки - 32 байта.

$$2^7 \cdot 2^5 = 2^{12} = 4 \text{ KB}$$

И таких карт тысячи…

Когда данных нет?

1.11 Алгоритм замены данных

1.11.1 Вероятностный метод

Выбираем ту ячейку которую перезаписать случайным образом.

А что если в цикле и уберут i?

1.11.2 FIFO

Первая добавленная и убирается. Добавляем нумерацию.

Но в случае i её снова уберут первой, ахахах.

1.11.3 LRU (Least Recently Used)

Добавить данные о последнем использовании.

Сколько бит добавить? Накладные расходы...

После выбора ячейки, обязательно нужно записать в память то, что мы удалили.

Другой процессор хочет прочитать удалённую, как?

- Bus snooping.
- Две технологии write_back и write_through.

1.12 Как обманывают потребителя?

3.2 ГГЦ - производительность памяти если передавать по 1му байту.

Шина 512 бит - 2^6 Байт.

$$20 \text{ МГц} \cdot 2^6 = 1280 \text{ МГц}$$

> **Как происходит сумма.**

2 Матрицы

2.1 Хранение матриц

2.1.1 Зачёт не получат

`a[i][j]` - выбор элемента.

```
double **a = new double*{n};
```

Два обращения к памяти выходит.

2.1.2 Есть шанс

```
a[i * n + j]
```

Делим массив на n участков по m элементов.

2.2 Перемножение матриц

Показать на таблице как идёт в памяти.

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

В A память читается строками, в B столбцами.

Если можно закешировать столбец b.

Поменяв порядок суммирования - i и j.

m - параметр.

$$n = k \cdot m + l$$

$$A = \begin{pmatrix} A_{11}^{m \times m} & \dots & A_{1k}^{m \times m} & A_{1,k+1}^{m \times l} \\ \vdots & \ddots & \ddots & \vdots \\ A_{k+1,1}^{l \times m} & \dots & \dots & A_{n+1,n+1}^{l \times l} \end{pmatrix}$$

$$C_{ij} = \sum_{s=1}^{k+1} A_{is} \cdot B_{sj}$$

Если поместятся в L1 кеш:

$$4 \cdot m^2$$

То мы из кеша обратились к данным:

$$2m^3$$

Даёт ускорение примерно в 10 - 12 раз.

$$C = A \cdot B$$
$$A = (A_{ij})$$
$$B = (B_{ij})$$
$$C = (C_{ij})$$

$$C_{ij} = \sum_{s=1}^{k} A_{is} \cdot B_{sj} + A_{ik+1} \cdot B_{k+1j}$$
$$C_{ik+1} = \sum A_{is}^{m \times m} \cdot B_{sk+1}^{m \times l} + A_{jk+1}^{m \times l} \cdot B_{k+1k+1}^{l \times l}$$
$$C_{k+1j} = \sum A_{k+1s}^{l \times m} \cdot B_{sj}^{m \times m} + A_{k+1k+1}^{l \times l} \cdot B_{k+1j}^{l \times m}$$
$$C_{k+1k+1} = \sum A_{k+1s}^{l \times m} \cdot B_{sk+1}^{m \times l} + A_{k+1k+1}^{l \times l} \cdot B_{k+1k+1}^{l \times l}$$

$c_{ij} = \sum_{q=1}^{k} a_{iq} \cdot b_{qj}$
$c_{ij+1} = \sum_{q=1}^{k} a_{iq} \cdot b_{qj+1}$
$c_{i+1j} = \sum_{q=1}^{k} a_{i+1q} \cdot b_{qj}$
L1 кеш - 32KB
4000 double
$\geq 3 \cdot m^2$
$\rightarrow$ m = 36 где-то
Блок делится на три, то есть при считывании, считываем сразу три элемента, так как общие данные

$$\begin{pmatrix} c_{ij} & c_{ij+1} & c_{ij+2} \\ c_{i+1j} & c_{i+1j+1} & c_{i+1j+2} \\ c_{i+2j} & c_{i+2j+1} & c_{i+2j+2} \end{pmatrix}$$

2.3 Методы

```
void mult (double *a, double *b, double *c, int n, int m)
  {
    int k, l, bl, i, j, r, h, r, t, ah;
    double *pc, *pa, *pb;
    k = n / m;
    l = n - k * m;
    bl = (l != 0 ? k + 1 : k);
    for (i = 0 ; i < bl ; i++)
      for (j = 0 ; j < bl ; j++)
        {
          // Вычисляем размер блока v x h
          v = (i * k ? m : l);
          h = (j < k ? m : l);
          pc = c + i * n * m + j * m;
          for (r = 0 ; r < v ; r++)
            for (t = 0 ; t < h ; t++)
              pc[r * n + t] = 0;

          for (s = 0 ; s < bl ; s++)
            {
              // A_is - v x ah, B_sj - ah x h
              ah = (s < k ? m : l);
              pa = a + i * n * m + s * m;
              pb = b + s * n * m + j * m;
```

```
            v3 = v % 3;
            h3 = h % 3;
            for (r = 0 ; r < v3 ; r++)
              {
                for (t = 0 ; t < h3 ; t++)
                  {
                    sum = 0;
                    for (q = 0 ; q < ah ; q++)
                      sum += pa[r * n + q] + pb[q * n + t];
                    pc[r * n + t] += sum;
                  }
                // Идём с шагом 3
                for (; t < h ; t += 3)
                  {
                    // 3 блока суммы
                    s00 = 0;
                    s01 = 0;
                    s02 = 0;

                    for (q = 0 ; q < ah ; q++)
                      {
                        s00 += pa[r * n + q] * pb[q * n + t];
                        s01 += pa[r * n + q] * pb[q * n + t + 1];
                        s02 += pa[r * n + q] * pb[q * n + t + 2];
                      }

                    pc[r * n + t] += s00;
                    pc[r * n + t + 1] += s01;
                    pc[r * n + t + 2] += s02;
                  }
              }

            for (; r < v ; r += 3)
              {
                for (t = 0 ; t < h3 ; t++)
                    {
                      s00 = 0;
                      s10 = 0;
                      s20 = 0;

                      for (q = 0 ; q < ah ; q++)
                        {
                          s00 += pa[r * n + q] * pb[q * n + t];
                          s10 += pa[(r + 1) * n + q] * pb[q * n + t];
                          s02 += pa[(r + 2) * n + q] * pb[q * n + t];
                        }

                      pc[r * n + t] += s00;
                      pc[(r + 1) * n + t] += s10;
                      pc[(r + 2) * n + t] += s20;
                    }

                  for (; t < h ; t += 3)
                    {
                      // Ставим параллельно, а не единым блоком равенств, показывая
компилятору, что они все независимы
```

```
                    s00 = 0;
                    s01 = 0;
                    s02 = 0;
                    s10 = 0;
                    s11 = 0;
                    s12 = 0;
                    s20 = 0;
                    s21 = 0;
                    s22 = 0;

                    for (q = 0 ; q < ah ; q++)
                      {
                        s00 += pa[r * n + q] * pb[q * n + t];
                        s01 += pa[r * n + q] * pb[q * n + t + 1];
                        s02 += pa[r * n + q] * pb[q * n + t + 2];
                        s10 += pa[(r + 1) * n + q] * pb[q * n + t];
                        s11 += pa[(r + 1) * n + q] * pb[q * n + t + 1];
                        s12 += pa[(r + 1) * n + q] * pb[q * n + t + 2];
                        s20 += pa[(r + 2) * n + q] * pb[q * n + t];
                        s21 += pa[(r + 2) * n + q] * pb[q * n + t + 1];
                        s22 += pa[(r + 2) * n + q] * pb[q * n + t + 2];
                      }

                    pc[r * n + t] += s00;
                    pc[r * n + t + 1] += s01;
                    pc[r * n + t + 2] += s02;
                    pc[(r + 1) * n + t] += s10;
                    pc[(r + 1) * n + t + 1] += s11;
                    pc[(r + 1) * n + t + 2] += s12;
                    pc[(r + 2) * n + t] += s20;
                    pc[(r + 2) * n + t + 1] += s21;
                    pc[(r + 2) * n + t + 2] += s22;
                  } // t
                } // r
              } // s
            } // j
          } // i
```

Мы имеем право написать два if, так как значительно уменьшили размер циклов...

```
if (a_(1 1) == 0)
```

Компилятор скажет вам, что он о вас думает

Потому что **нельзя** сравнивать дробные числа равенством, но:

```
if (|a_(1 1)| > eps)
```

2.3.1 Норма матрицы

1. $\|A\| \geq 0, \|A\| = 0 \Leftrightarrow A = 0$
2. $\|\lambda A\| = |\lambda| \cdot \|A\|$
3. $\|A + B\| \leq \|A\| + \|B\|$
4. $\|A \cdot B\| \leq \|A\| \cdot \|B\|$

2.3.1.1 Примеры:

1. $\|A\| = \sup_{x \neq 0} \left(\|A \cdot x\| \frac{1}{\|x\|} \right)$
2. $\|A\| = \left(\sum_{i=1}^{n} \sum_{j=1}^{n} a_{ij} \right)^{\frac{1}{2}}$ - не норма, потому что норма единичной матрицы - корень n, а должен быть 1

3. $\max_{\|x\|=1}\|Ax\|$

2.3.2 Проверка решения системы

$$\frac{\|x_* - x\|}{\|x_*\|} \leq x(A) \cdot \frac{\|Ax - b\|}{\|b\|}$$

Не успел записать

Получили, что для некоторых матриц, метод не работает, и ошибка практически 100%

2.3.3 Решения

2.3.3.1 Метод Гаусса

Есть главный элемент, мы его выбираем наибольшим т.к. тогда при делении на него - размер ошибки предыдущих шагов не увеличивается!

2.3.3.2 Блочный метод Гаусса

Разделить матрицу на блоки и работать поблочно

То есть теперь мы работаем не с числами, а полноценными матрицами

2.3.3.3 Метод Жордана

Ясненько...

Легче матрицу вычислить, чем её прочитать

Если две матрицы ниль и верх треугольные, то лучше их хранить в одном

$$\begin{pmatrix} (A_{11})_{11} & \dots & (A_{11})_{1m} & (A_{12})_{11} & \dots & (A_{1k+1})_{1l} \\ \dots & & & & & \end{pmatrix}$$

get_block - копирует всю матрицу put_block - возвращает

Богачёв К.Ю. Практикум на ЭВМ. Методы решения линейных систем и нахождения собственных значений.

2.4 Отчёт

В формате LaTEX

report.tex

2.4.1 Управляющие команды

Начинаются с \

```
\begin {document}

\end {document}
```

Шапка нужна для того чтобы задать стиль

Стиль:
1. Article - наш случай
2. Report
3. Book

Шрифт:
1. Растровый (Пиксельный)
2. Векторный (Кривыми задан)

Кодировка:
- UTF-8 (другие не надо)

bfseries - (bold font)

2.4.1.1 В порядке возрастания

1. large
2. Large
3. LARGE
4. huge

2.4.2 Математические формулы

$ $ - инлайн формулы

$$ $$ - На насколько строк

Состояние потоков:
1. остановлен
2. терминирован - завершил работу и ждёт
3. ждёт
4. готов - есть всё для работы, кроме лог. процессора
5. выполняется - появился лог. процессор

Если процессор 1 и в каждый момент времени выполняется 1 штука, то это не параллельно, поэтому в многозадачных системах это не так

Система многозадачная если может быть более одного готового потока

$f_c = 2.4$ HGz $f_p = 100$ Гц

2.4.2.1 Типы планирования

1. Приоритетное - это число приписанное потоку операционной системой. Приоритеты бывают:
   1. фиксированные (задано разработчиком) - известны все процессы
   2. динамические - меняются в течение работы ОС

   Правила приоритизации:
   1. Unix Style
      1. готов -> выполняется
      2. Получает максимальный приоритет
      3. Если использовал всё свободное время - приоритет понижается
      4. Если с каждым запуском так происходит, приоритет падает до дна.

      Работает отлично

      Что если три потока и два процесса Тогда два варианта:
      1. Либо разделяем каждому по

$$\frac{1}{3}$$

      2. Либо одному потоку 1 целый процесс, а остальным по половинке другого процесса

LWP - light weight proof? Thread - поток исполнения

Граф состояний потока: не существует -> остановлен / не обслуживается -> готов -> выполняется -> ожидает ресурса / времени / события (нажатие клавиши) могут переходить друг в друга

Стек (Stack) - временная область в памяти
1. сохраняет адрес возврата при вызове функции, а также регистры которые используются и могут меняться вызываемой функцией (пример i)
2. могут меняться локальные переменные
3. сохраняется контекст = набор регистров, которые может менять поток и которые полностью определяют его состояние - это значит что при смене потока и возвращении к нему - это будет незаметно для самого потока

Изоляция процессов позволяет при Seg Fault не сломать другой процесс, так как одному процессу нет доступа ко второму

Это не легко

Потребовались:

2.4.2.2 Методы межпроцессорного взаимодействия

1. Разделяемая память - память доступная для

$$> 1$$

   процесса, если блок доступен и используется несколькими потоками

Концепции:
1. Семафор (светафор на железной дороге) Ячейка памяти доступна > 1 процессу:
   - s = 0 - занята
   - s = 1 - свободна (и наоборот)

   if (s != 0) { s++; s–; }

А если кто-то меняет семафор, то ему тоже нужен семафор и пошло поехало… Поэтому s недоступен напрямую потокам, это происходит через процессор

Интересный факт, раньше это была просто ячейка, которая даже не сохраняет владельца

Поэтому поток мог сначала освободить ячейку, а потом занять…

сигналы - асссинхронные события (тот, кто хочет получить сигнал - реогистрирует указатель на функцию, которая будет вызвана при получении данного события, тот кто хочет отправить сигнал - отправляет) события - это события

signal (функция)

Всего 64 сигнала

30 заняты:

1. 11 - Seg. Fault
2. – - Bus Error
3. 8 - FPE
4. 6 - Abort
5. – - Seg. Pipe (Если кто-то пишет в канал и его собираются закрыть)
6. 9 - killed (Ему нельзя поставить signal - один такой)
7. 15 - terminate (Чтобы программа завершилась)

Поток:

1. Создать
2. Открыть -> нет -> Создать

```
#include <sys/ipc.h> // inter proccess communication
#include <sys/shm.h> // shares memory

shmget (); // создать / открыть

shmat (); // Присоединить к адресному пространству, at - attach

// Адресов быть не должно в общих данных, так как в адресных пространствах разные
адреса

shmdt (); // Отсоединить

shmctl (); // control, можно давать в него разные команды, в том числе и удалить
```

3 Семинар 7. Параллельное программирование

3.1 Создание потока

Поток — последовательность выполнения команд внутри процесса. Первый поток процесса начинает работу с функции `main`. Для каждого дополнительного потока задаётся функция, с которой он начнёт выполнение.

```
#include <pthread.h>

int pthread_create (
  pthread_t *tid,
  const pthread_attr_t *attr,
  void *(*start_routine) (void *),
  void *arg
);
```

- `tid` — идентификатор созданного потока;
- `attr` — атрибуты потока; значение `nullptr` задаёт атрибуты по умолчанию;
- `start_routine` — функция потока;
- `arg` — аргумент, передаваемый функции потока.

При создании потока:

1. создаётся стек потока;
2. создаётся состояние потока;
3. в новом потоке вызывается `start_routine(arg)`.

Поток может быть присоединяемым (`joinable`) или отсоединённым (`detached`). Завершения присоединяемого потока можно дождаться с помощью `pthread_join`.

3.2 Завершение потока

```
int pthread_join (pthread_t tid, void **ret_val);
void pthread_exit (void *ret_val);
```

- `pthread_join` ожидает завершения потока `tid` и позволяет получить возвращённое им значение;
- `return` из `start_routine` завершает текущий поток;
- `pthread_exit` завершает вызвавший его поток;
- `exit` и `abort` завершают весь процесс вместе со всеми его потоками.

Если поток завершился до вызова `pthread_join`, его результат сохраняется до присоединения.

3.2.1 Пример создания потоков

Основной поток выполняет работу для номера `0`, а остальные потоки создаются с помощью `pthread_create`.

```
#include <pthread.h>
#include <stdio.h>

void * thread_func (void *arg)
{
  int k = *((int *)arg);
  printf ("Hello from thread %d\n", k);
  return nullptr;
}

// ./a.out 5 — запустить вычисление в пяти потоках.
int main (int argc, char *argv[])
{
  int *args = nullptr;
  pthread_t *tids = nullptr;
```

```
  int p, k;

  if (argc != 2 || sscanf (argv[1], "%d", &p) != 1)
    {
      printf ("usage: %s p\n", argv[0]);
      return 1;
    }

  args = new int[p];
  tids = new pthread_t[p];

  if (args == nullptr || tids == nullptr)
    {
      // Обработка ошибки выделения памяти.
    }

  for (k = 0; k < p; ++k)
    args[k] = k;

  for (k = 1; k < p; ++k)
    {
      if (pthread_create (tids + k, nullptr, thread_func, args + k))
        {
          // Обработка ошибки создания потока.
        }
    }

  thread_func (args + 0);

  for (k = 1; k < p; ++k)
    pthread_join (tids[k], nullptr);

  delete[] args;
  delete[] tids;

  return 0;
}
```

3.3 Динамическая память в многопоточной программе

Ранние реализации `malloc` и `free` создавались для однопоточных программ и не обеспечивали безопасный одновременный доступ из нескольких потоков. После появления многопоточных систем аллокаторы стали защищать общие структуры синхронизацией.

Современные реализации `malloc` и `free` являются потокобезопасными, но одновременное выделение памяти может быть дорогой операцией:

- общий аллокатор может использовать взаимное исключение;
- локальные арены и кэши потоков уменьшают конкуренцию за общие структуры;
- локальные арены могут потребовать дополнительную память.

3.4 Передача аргументов через класс

Класс аргументов позволяет передать функции потока несколько связанных значений через один указатель.

```
class Args
{
public:
```

```
  int a;
  double *a;
};
```

3.5 Взаимное исключение

Критическая секция — участок программы, в котором поток работает с общими изменяемыми данными. Одновременно критическую секцию должен выполнять только один поток.

3.5.1 Семафор

Семафор — синхронизационный объект со счётчиком доступных ресурсов. Он позволяет ограничить число потоков, одновременно использующих ресурс, но обычно не хранит владельца.

Работа с семафором может требовать системных вызовов и переключения состояния потока.

3.5.2 Мьютекс

Мьютекс (`mutual exclusion`) — объект взаимного исключения, который допускает в критическую секцию только один поток.

Одна из возможных моделей реализации мьютекса содержит состояние захвата и идентификатор владельца. Для изменения состояния используются неделимые, или атомарные, инструкции процессора. Конкретная реализация не обязана содержать отдельный объект семафора.

```
if (m.s > 0)
  m.s = 0;
else
  wait_until (m.s > 0);
```

Проверка и изменение `m.s` должны выполняться как одна атомарная операция, иначе два потока могут одновременно увидеть свободное состояние.

3.5.3 Защита общей переменной

```
int f1 ()
{
  static mutex m;
  static int x = 0;

  m.lock ();
  x++;
  m.unlock ();

  return x;
}
```

Один и тот же мьютекс должен защищать все обращения к общей переменной `x`.

3.5.4 Повторный захват мьютекса

```
class args
{
public:
  int x;
  mutex m;
};

void f1 (args *a)
{
  a->m.lock ();
```

```
  a->x++;
  a->m.unlock ();
}

void f2 (args *a)
{
  a->m.lock ();
  f1 (a);
  a->m.unlock ();
}
```

Функция `f2` захватывает `a->m`, после чего `f1` пытается захватить тот же мьютекс повторно. Для обычного нерекурсивного мьютекса это приводит к взаимной блокировке.

3.6 Параллельный блочный алгоритм

3.6.1 Собственная память потока

Данные распределяются между потоками так, чтобы каждый поток преимущественно работал со своей областью памяти. Это уменьшает число обращений к общим изменяемым данным и количество точек синхронизации.

3.6.2 NUMA

NUMA (`non-uniform memory access`) — архитектура, в которой скорость доступа к памяти зависит от её расположения относительно процессора.

Размещение памяти по правилу первого обращения:

1. выделить общую память с помощью `new`;
2. создать потоки с помощью `pthread_create`;
3. привязать поток `k` к выбранному ядру с помощью `set_affinity`;
4. выполнить из потока `k` первое обращение к предназначенной ему области памяти.

При политике первого обращения страницы памяти размещаются в NUMA-узле потока, который первым к ним обратился.

3.6.3 Кэш

Если несколько потоков читают одни и те же данные, они конкурируют за пропускную способность памяти и место в кэше. При записи в общую строку кэша дополнительно возникает обмен сообщениями протокола когерентности.

Поэтому данные следует разделять между потоками, а готовые строки передавать остальным потокам один раз для последующего чтения.

3.6.4 Требования к отчёту

- описать разделение данных и собственную область памяти каждого потока;
- указать формулы границ работы потока `k`;
- перечислить точки синхронизации и свести их число к минимуму;
- оценить вычислительную сложность:

$$\frac{C_1 n^3 + C_2 n^2 m + C_3 n m^2 + C_n m^3}{p};$$

- оценить число точек синхронизации:

$$C\frac{n}{m}.$$

Каждый поток должен самостоятельно вычислять строки, от которых зависит его дальнейшая работа. Это уменьшает число обменов данными и точек синхронизации.

4 Семинар 8. Средства синхронизации POSIX Threads

4.1 Мьютекс

Мьютекс — объект взаимного исключения, защищающий критическую секцию от одновременного выполнения несколькими потоками.

```
#include <pthread.h>

pthread_mutex_t m;

int pthread_mutex_init (
  pthread_mutex_t *m,
  const pthread_mutexattr_t *attr
);

int pthread_mutex_destroy (pthread_mutex_t *m);
int pthread_mutex_lock (pthread_mutex_t *m);
int pthread_mutex_unlock (pthread_mutex_t *m);
int pthread_mutex_trylock (pthread_mutex_t *m);
```

- `pthread_mutex_init` инициализирует мьютекс; значение `nullptr` задаёт атрибуты по умолчанию;
- `pthread_mutex_destroy` уничтожает инициализированный и незахваченный мьютекс;
- `pthread_mutex_lock` захватывает мьютекс или ожидает его освобождения;
- `pthread_mutex_unlock` освобождает мьютекс;
- `pthread_mutex_trylock` пытается захватить мьютекс без ожидания.

Статическая инициализация выполняется до начала работы потоков:

```
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
```

Один объект нельзя одновременно инициализировать из нескольких потоков с помощью `pthread_mutex_init`. Статический инициализатор создаёт мьютекс заранее и исключает такую гонку.

4.2 Условная переменная

Условная переменная (`condition variable`) позволяет потоку ожидать изменения состояния общих данных. Само условие хранится в данных программы и проверяется под мьютексом.

```
int pthread_cond_init (
  pthread_cond_t *c,
  const pthread_condattr_t *attr
);

int pthread_cond_signal (pthread_cond_t *c);
int pthread_cond_broadcast (pthread_cond_t *c);
int pthread_cond_destroy (pthread_cond_t *c);
int pthread_cond_wait (pthread_cond_t *c, pthread_mutex_t *m);

static pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

- `pthread_cond_signal` пробуждает один ожидающий поток;
- `pthread_cond_broadcast` пробуждает все ожидающие потоки;
- `pthread_cond_wait` атомарно освобождает мьютекс и переводит поток в ожидание; перед возвратом функция снова захватывает мьютекс.

Условие проверяется в цикле, поскольку после пробуждения оно могло снова стать ложным.

```
pthread_mutex_lock (&m);

while (!x)
  pthread_cond_wait (&c, &m);

pthread_mutex_unlock (&m);
```

4.2.1 Пример ожидания счётчика

```
void f1 (int p)
{
  pthread_mutex_lock (&m);

  while (x != p)
    pthread_cond_wait (&c, &m);

  pthread_mutex_unlock (&m);
}

void f2 ()
{
  pthread_mutex_lock (&m);
  x++;
  pthread_cond_signal (&c);
  pthread_mutex_unlock (&m);
}
```

Поток `f1` ожидает, пока общий счётчик `x` не достигнет `p`. Поток `f2` изменяет счётчик под тем же мьютексом и уведомляет ожидающий поток.

4.3 Суммирование результатов потоков

Функция `reduce_sum` выполняет коллективное суммирование массивов. Каждый из `p` потоков передаёт свой массив, после чего итоговая сумма копируется всем участникам.

Два счётчика образуют две фазы синхронизации:

- `t_in` считает потоки, передавшие данные;
- `t_out` считает потоки, получившие результат.

```
void reduce_sum (int p, double *a = nullptr, int n = 0)
{
  static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
  static pthread_cond_t c_in = PTHREAD_COND_INITIALIZER;
  static pthread_cond_t c_out = PTHREAD_COND_INITIALIZER;
  static int t_in = 0;
  static int t_out = 0;
  static double *res = nullptr;

  if (p <= 1)
    return;

  pthread_mutex_lock (&m);

  if (res == nullptr)
    res = a;
  else
    for (int i = 0; i < n; ++i)
      res[i] += a[i];
```

```
  t_in++;

  if (t_in >= p)
    {
      t_out = 0;
      pthread_cond_broadcast (&c_in);
    }
  else
    {
      while (t_in < p)
        pthread_cond_wait (&c_in, &m);
    }

  if (res != a)
    for (int i = 0; i < n; ++i)
      a[i] = res[i];

  t_out++;

  if (t_out >= p)
    {
      res = nullptr;
      t_in = 0;
      pthread_cond_broadcast (&c_out);
    }
  else
    {
      while (t_out < p)
        pthread_cond_wait (&c_out, &m);
    }

  pthread_mutex_unlock (&m);
}
```

Вторая фаза не позволяет начать следующий вызов `reduce_sum`, пока результат текущего вызова не получили все потоки.

4.4 Барьер

Барьер — точка синхронизации, в которой каждый поток ожидает прибытия остальных участников.

```
int pthread_barrier_init (
  pthread_barrier_t *b,
  const pthread_barrierattr_t *attr,
  unsigned int count
);

int pthread_barrier_wait (pthread_barrier_t *b);
int pthread_barrier_destroy (pthread_barrier_t *b);

static pthread_barrier_t b;
```

```
pthread_barrier_wait (&b);
```

После прибытия `count` потоков все они продолжают выполнение. Одному потоку `pthread_barrier_wait` возвращает `PTHREAD_BARRIER_SERIAL_THREAD`, остальным — `0`.

4.5 Измерение времени

Полное время — фактическое время между началом и окончанием участка программы.

```
#include <sys/time.h>

double get_full_time (void)
{
  struct timeval buf;
  gettimeofday (&buf, nullptr);

  return buf.tv_sec + buf.tv_usec / 1.e6;
}
```

Процессорное время — время, в течение которого процессор выполнял команды программы. В зависимости от используемой функции оно может измеряться для одного потока или для всего процесса.

`getrusage` с параметром `RUSAGE_THREAD` возвращает статистику текущего потока.

```
#include <sys/resource.h>

double get_cpu_time (void)
{
  struct rusage buf;
  getrusage (RUSAGE_THREAD, &buf);

  return buf.ru_utime.tv_sec + buf.ru_utime.tv_usec / 1.e6;
}
```

4.6 Аргументы потока

Объект `Args` хранит общие массивы, параметры задачи, номер потока и результаты измерений.

```
class Args
{
public:
  double *a = nullptr;
  double *b = nullptr;
  double *x = nullptr;

  const char *name = nullptr;
  int n = 0;
  int m = 0;
  int p = 0;
  int k = 0;
  int s = 0;

  double t = 0;
  double t_full = 0;

  Args () = default;
  Args (const Args &) = delete;
  Args (Args &&) = default;
  Args & operator= (const Args &) = delete;
  Args & operator= (Args &&) = default;
  ~Args () = default;
};
```

Копирование запрещено, поэтому объект аргументов не может быть случайно скопирован вместе с указателями на общие данные.

4.7 Создание потоков

Для каждого потока создаётся отдельный объект `Args`. Основной поток выполняет функцию с номером `0`, остальные запускаются через `pthread_create`.

```
int main (int argc, char *argv[])
{
  double *a = nullptr;
  double *b = nullptr;
  double *x = nullptr;
  Args *ap = nullptr;
  pthread_t *tid = nullptr;

  int n = 0, m = 0, p = 0, s = 0;
  const char *name = nullptr;

  // Разбор аргументов командной строки.

  ap = new Args[p];
  tid = new pthread_t[p];

  if (ap == nullptr || tid == nullptr)
    {
      // Обработка ошибки выделения памяти.
    }

  a = new double[n * n];
  b = new double[n];
  x = new double[n];

  for (int u = 0; u < p; ++u)
    {
      ap[u].a = a;
      ap[u].b = b;
      ap[u].x = x;
      ap[u].name = name;
      ap[u].n = n;
      ap[u].m = m;
      ap[u].p = p;
      ap[u].k = u;
      ap[u].s = s;
    }

  for (int u = 1; u < p; ++u)
    {
      if (pthread_create (tid + u, nullptr, thread_func, ap + u))
        {
          // Обработка ошибки создания потока.
        }
    }

  thread_func (ap + 0);

  for (int u = 1; u < p; ++u)
```

```
    pthread_join (tid[u], nullptr);

  // Вывод результата и освобождение памяти.
  return 0;
}
```

4.8 Функция потока

4.8.1 Привязка к процессору

Привязка потока к логическому процессору делает распределение вычислений предсказуемым и позволяет применять политику первого обращения в NUMA-системе.

```
void * thread_func (void *ptr)
{
  Args *ap = (Args *)ptr;
  double *a = ap->a;
  double *b = ap->b;
  double *x = ap->x;
  const char *name = ap->name;
  int n = ap->n;
  int m = ap->m;
  int p = ap->p;
  int k = ap->k;
  int s = ap->s;

  cpu_set_t cpu;
  CPU_ZERO (&cpu);

  int n_cpus = get_nprocs ();
  int cpuid = n_cpus - 1 - (k % n_cpus);

  CPU_SET (cpuid, &cpu);
  pthread_t tid = pthread_self ();
  pthread_setaffinity_np (tid, sizeof (cpu), &cpu);
```

4.8.2 Первое обращение к памяти

Каждый поток обнуляет назначенные ему блоки. Это обеспечивает первое обращение к соответствующим страницам памяти из нужного потока.

```
  for (int i = k * m; i < n; i += p * m)
    {
      int h = (i + m < n ? m : n - i);

      memset (a + i * n, 0, h * n * sizeof (double));
      memset (b + i, 0, h * sizeof (double));
      memset (x + i, 0, h * sizeof (double));
    }
```

4.8.3 Подготовка и решение задачи

Если задано имя файла, матрицу читает поток `0`, а результат чтения распространяется между всеми потоками. Иначе каждый поток инициализирует свою часть матрицы по формуле.

```
  double *block = new double[m * m];
  double res = 0;

  if (name != nullptr)
    {
```

```
      if (k == 0)
        res = read_matrix (a, n, name);

      reduce_sum (p, &res, 1);

      if (res < 0)
        {
          delete[] block;
          return nullptr;
        }
    }
  else
    {
      init_matrix (a, n, s, k, p);
    }

  init_b (a, n, b, k, p);

  double t_full = get_full_time ();
  double t = get_cpu_time ();

  solve (a, b, x, n, m, k, p);

  t = get_cpu_time () - t;
  t_full = get_full_time () - t_full;

  ap->t = t;
  ap->t_full = t_full;

  if (k == 0)
    {
      // Вывод результата.
    }

  delete[] block;
  reduce_sum (p);
  return nullptr;
}
```

Вызов `reduce_sum(p)` без массива используется как точка синхронизации перед завершением потоков.

4.9 Функции параллельного алгоритма

```
void init_matrix (
  double *a,
  int n,
  int s,
  int k,
  int p
)
{
}

void matrix_mult_vector (
  double *a,
```

```
  double *x,
  double *b,
  int n,
  int m,
  int k,
  int p
)
{
}
```

`init_matrix` инициализирует часть матрицы, назначенную потоку `k`. `matrix_mult_vector` распределяет между потоками умножение матрицы на вектор.